



PLATFORM OVERVIEW

Twitter (X) - System Architecture Analysis

DartX Innovators

Team Meet · 19 June 2026 · 6:30 PM · Google Meet

1. Problem It Solves

- 🌐 Enables real-time communication and global information sharing at scale.
- 📢 Allows users to post text updates, news, opinions, images, and videos to a public feed.
- 💬 Facilitates global conversations through likes, comments, reposts, and hashtag-based discovery.
- 🌍 Provides a platform connecting billions of people across the world in real time.

2. Frontend Components

#	Component	Description
01	Login / Register Page	Entry point for new and returning users with authentication forms
02	Home Feed	Chronological and algorithmic timeline of posts from followed accounts
03	Create Post / Tweet	Rich text composer supporting text, images, videos, polls, and GIFs
04	User Profile	Displays user bio, post history, followers, and following counts
05	Notifications	Real-time alerts for likes, mentions, reposts, follows, and DMs
06	Search Bar	Full-text and hashtag search across users, posts, and trending content
07	Comments Section	Threaded replies under each post with nested conversation support
08	Direct Messages	Private encrypted one-on-one and group messaging interface
09	Settings Page	Account, privacy, security, notification, and display preferences
10	Trending Topics	Curated real-time list of hashtags and topics grouped by region

3. Backend Components

#	Service / Component	Responsibility
01	API Gateway	Single entry point that routes all client requests to appropriate microservices
02	Authentication Service	Handles login, registration, session management, OAuth, and JWT issuance
03	User Service	Manages user profiles, followers, following relationships, and preferences

04	Post / Tweet Service	CRUD operations for posts; manages feeds, reposts, and content visibility
05	Comment Service	Handles threaded comments and nested reply trees for each post
06	Like Service	Records, counts, and broadcasts like events for posts and comments
07	Notification Service	Generates and dispatches real-time in-app and push notifications
08	Media Service	Handles upload, compression, CDN delivery of images and videos via S3
09	Analytics Service	Aggregates engagement metrics, trending data, and platform usage insights
10	Message Queue (Kafka)	Decouples services using event-driven async communication for resilience

4. Technologies Used

Layer	Technology Options	Purpose
 Frontend	Vue / React	Reactive UI frameworks for building a fast, component-based SPA
 Backend	Node.js / Go	High-throughput event-driven (Node) or concurrent (Go) API services
 Database	PostgreSQL / MySQL	Relational RDBMS for structured user, post, and interaction data
 Caching	Redis	In-memory key-value store for feeds, sessions, and rate limiting
 Queue	Kafka	Distributed message streaming for async, fault-tolerant event processing
 Storage	Amazon S3	Scalable object storage for user-uploaded images and videos
 Authentication	OAuth / JWT	Secure delegated authorization and stateless token authentication
 Deployment	Kubernetes	Container orchestration for auto-scaling, rolling deploys, and HA

5. Database Requirements

5.1 Users Table

Column	Type / Key	Description
<code>user_id</code>	 Primary Key	Unique identifier for each user account
<code>username</code>	VARCHAR, UNIQUE	Unique public-facing handle for the user
<code>email</code>	VARCHAR, UNIQUE	User's email address for login and notifications
<code>password_hash</code>	VARCHAR	Bcrypt-hashed password for secure authentication
<code>created_at</code>	TIMESTAMP	Account creation timestamp with timezone support

5.2 Posts Table

Column	Type / Key	Description
<code>post_id</code>	 Primary Key	Unique identifier for each post
<code>user_id</code>	 Foreign Key	References the author from the Users table
<code>content</code>	TEXT / MEDIUMTEXT	Post body — text, media URL, or a combination
<code>created_at</code>	TIMESTAMP	Post creation timestamp with timezone support
<code>likes_count</code>	INT, DEFAULT 0	Denormalized running count of likes for performance
<code>comments_count</code>	INT, DEFAULT 0	Denormalized running count of comments for performance

5.3 Comments Table

Column	Type / Key	Description
<code>comment_id</code>	 Primary Key	Unique identifier for each comment
<code>post_id</code>	 Foreign Key	References the parent post in the Posts table
<code>user_id</code>	 Foreign Key	References the commenter in the Users table
<code>comment</code>	TEXT	The body of the user's comment
<code>created_at</code>	TIMESTAMP	Comment creation timestamp with timezone support

5.4 Followers Table

Column	Type / Key	Description
<code>follower_id</code>	 Foreign Key	The user who initiates the follow action
<code>following_id</code>	 Foreign Key	The user being followed; pair forms the unique PK

5.5 Notifications Table

Column	Type / Key	Description
<code>notification_id</code>	 Primary Key	Unique identifier for each notification record
<code>user_id</code>	 Foreign Key	The recipient user from the Users table
<code>message</code>	TEXT	Human-readable notification message content
<code>status</code>	ENUM	Read / Unread status flag for UI badge management

6. APIs Overview

6.1 External APIs

-  Firebase Cloud Messaging — Push notification delivery to iOS and Android devices
-  Google Maps API — Location tagging and geo-tagged post metadata
-  Cloud Storage API — Scalable image and video upload and CDN delivery

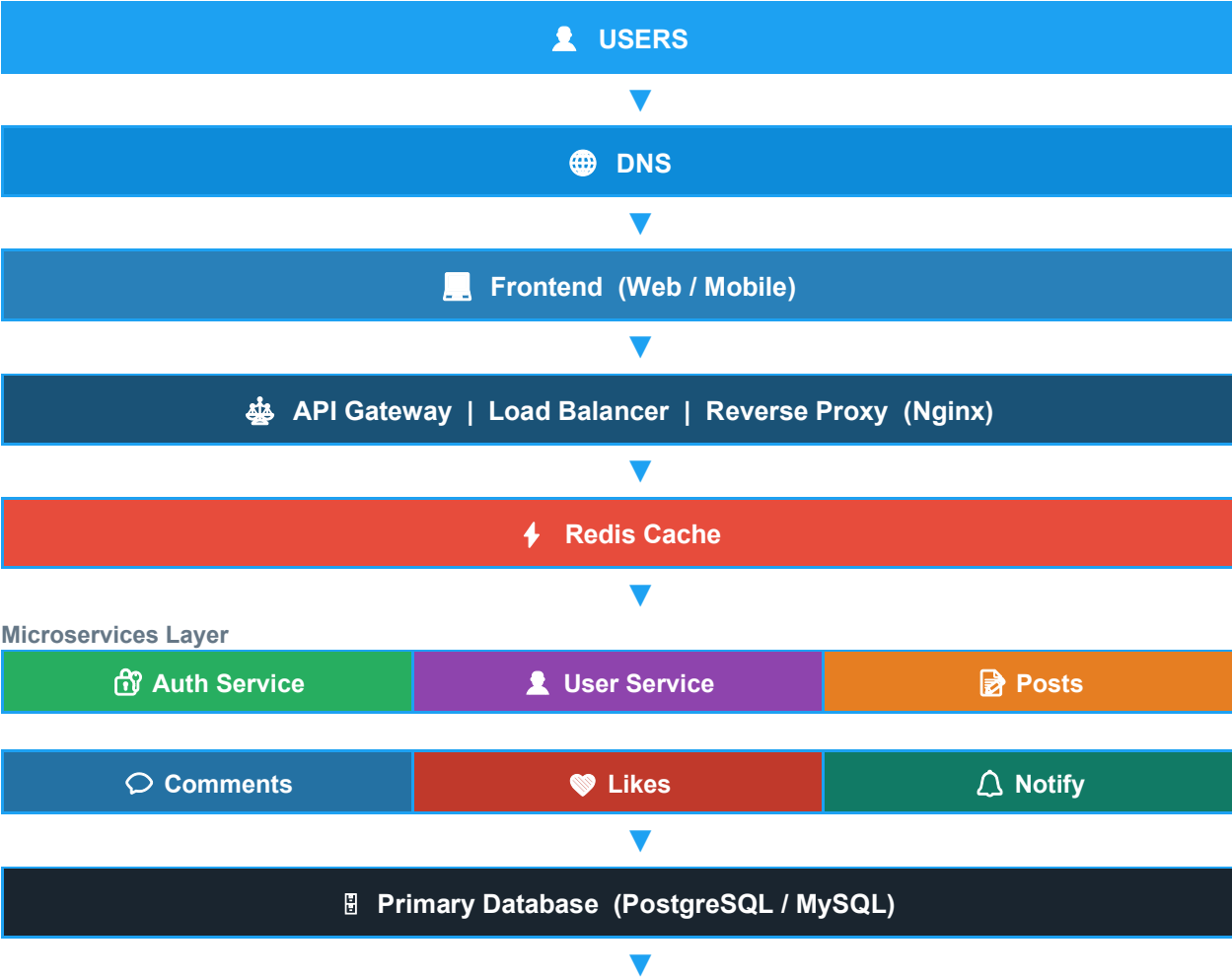
6.2 Internal REST API Endpoints

Method	Endpoint	Description
POST	<code>/login</code>	Authenticates a user and returns a JWT access token
POST	<code>/register</code>	Creates a new user account with hashed credentials
POST	<code>/posts</code>	Creates a new post with text and optional media attachments
GET	<code>/posts</code>	Retrieves paginated list of all posts for the home feed

GET	/posts/:id	Fetches a specific post by its unique identifier
POST	/comments	Adds a threaded comment to a specified post
POST	/likes	Toggles a like on a specified post by the authenticated user
GET	/notifications	Fetches all notifications for the authenticated user
GET	/profile	Returns the profile of the currently authenticated user
GET	/profile/:id	Fetches the public profile of any user by their ID

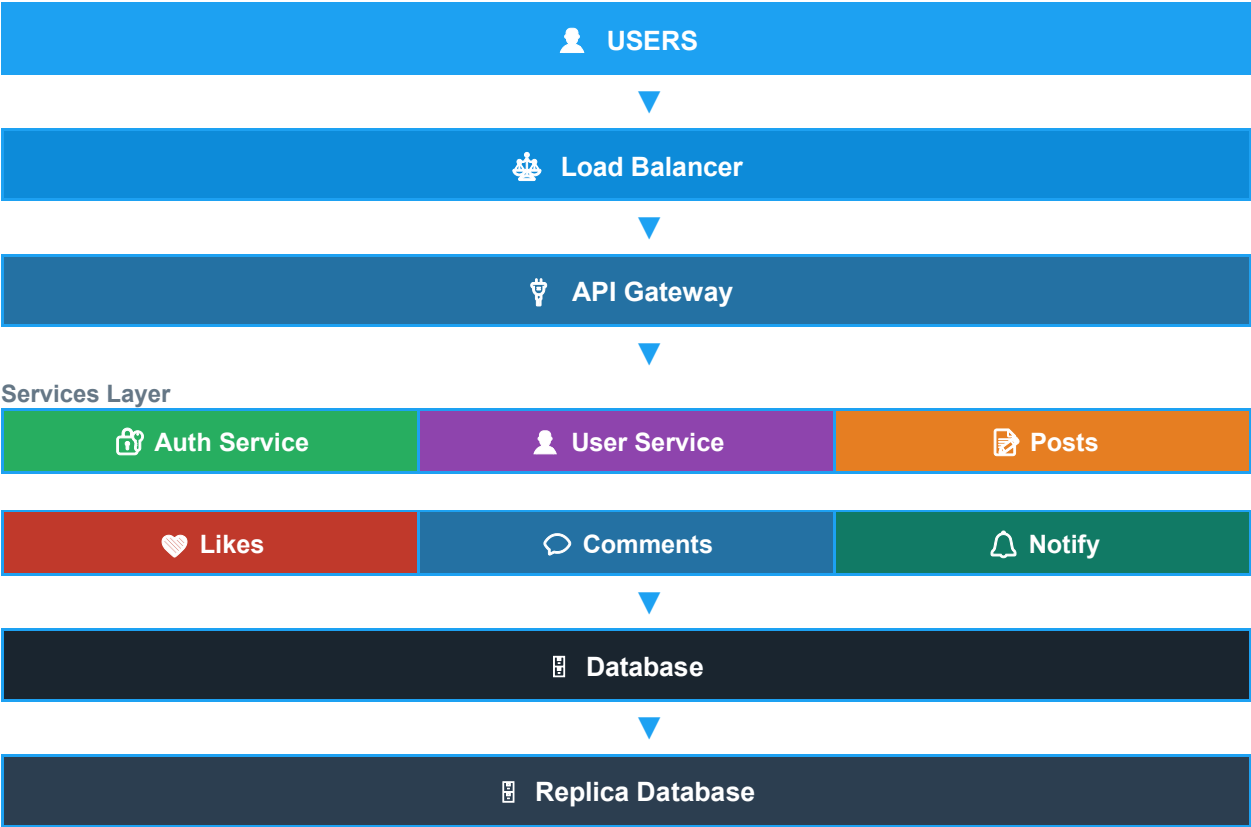
7. Architecture Diagrams

High-Level Architecture Diagram





Cloud Infrastructure Diagram



8. Security Features

✓	Security Control	Implementation Detail
	Password Hashing	Passwords stored using bcrypt with salt rounds — never in plain text
	HTTPS Encryption	All client-server communication encrypted via TLS 1.3 certificates
	Two-Factor Authentication	TOTP or SMS-based second factor for high-risk login scenarios
	End-to-End Encryption	DMs encrypted at rest and in transit — server cannot read content

	Input Validation	Server-side sanitization and schema validation on all API inputs
	SQL Injection Prevention	Parameterized queries and ORM usage; no raw string interpolation
	JWT Authentication	Stateless, signed, short-lived tokens with refresh token rotation
	OAuth Integration	Delegated authorization via OAuth 2.0 for third-party app access

9. Team Contributions

Team Member	Contribution
Hasini Chandolu	Research on Twitter (X) platform and the core problem it solves
Premavathy	Design and documentation of all frontend UI components
Divya	Architecture and documentation of all backend microservices
Karthikeya Volety	Database schema design and API specifications
Tamil Selvan	Security features, threat modelling, and encryption strategy

10. Summary

This architecture outlines a scalable, secure, and efficient design for a Twitter (X)-like platform. It integrates modern technologies for real-time communication, robust microservices-based backend services, and a responsive frontend — ensuring a seamless user experience and high system reliability. The system is built to handle millions of concurrent users with horizontal scalability via Kubernetes, low latency through Redis caching, asynchronous event processing via Kafka, and a strong security posture employing end-to-end encryption, JWT authentication, and OAuth integration.

- ◆ Scalability achieved through Kubernetes-orchestrated microservices and Redis caching layers.
- ◆ Resilience ensured via Kafka event streaming and database replication with read replicas.
- ◆ Security hardened with JWT, OAuth 2.0, bcrypt hashing, E2E encryption, and input validation.
- ◆ Extensibility supported through clean REST API contracts and service decoupling.